

AI-Enhanced Fast Cosmological Simulations for Accurate Large-Scale Structure Inference

David Amrein

July 7, 2026

Contents

1	Introduction	3
2	Theoretical Background	3
2.1	Cosmological Framework	3
2.1.1	The Cosmic Web	3
2.1.2	Matter Power Spectrum and Transfer Function	4
2.1.3	Lightcones and Snapshots	5
2.2	Cosmological Simulations	6
2.2.1	DISCO-DJ	6
2.2.2	Pkdgrav	7
2.2.3	CosmoGridV1	8
3	Methods and implementation	8
3.1	Pipeline Architecture	8
3.2	IC generation	9
3.2.1	Validation against CosmoGridV1	9
3.2.2	Backscaled initial conditions and hand-off to DISCO	9
3.3	Shell Builder	10
3.3.1	Core	10
3.3.2	Crossing Condition	11
3.4	Neural Network	13
3.4.1	Dimensional Reduction	13
3.4.2	Data preparation	14
3.4.3	Network Architecture	14
3.4.4	Training Procedure	14
3.4.5	Validation Strategy	14
4	Computational Infrastructure	14
4.1	Computing Environments	14
4.1.1	Local Development	14
4.1.2	Euler (ETH Zürich)	15

4.1.3	CSCS (Alps / Piz Daint)	15
4.2	Software Engineering	15
4.2.1	Code Structure	15
4.2.2	Job Scheduling and Submission	15
4.2.3	Version Control (GitHub)	16
5	Results	16
6	Conclusion	17

Abstract

1 Introduction

2 Theoretical Background

2.1 Cosmological Framework

2.1.1 The Cosmic Web

On scales of a few to a few hundred megaparsecs, matter in the Universe is not smoothly distributed but arranged into an interconnected network of dense compact knots, elongated filaments, two-dimensional sheets, and large underdense voids – collectively known as the *cosmic web* [2, 13]. The term originates with Bond, Kofman, and Pogosyan, who showed that this filament-dominated pattern is already present in embryonic form in the tidal field of the initial density perturbations, with the nonlinear gravitational evolution sharpening rather than creating it [2]. The same gravitational instability that seeds this pattern is responsible for its growth: initially small density fluctuations are amplified by their own self-gravity, so that overdense regions contract while voids expand and come to dominate the volume of the Universe [2]. Large galaxy redshift surveys – 2dFGRS, SDSS, and the 2MASS redshift survey (2MRS) among them – as well as cosmological N -body simulations both recover this same web-like arrangement of galaxies and dark matter [13].

Gravitational instability drives the emergence of anisotropic structure: because collapse along the direction of a region’s strongest tidal compression proceeds fastest, an initially only mildly non-spherical overdensity first flattens into a sheet, then contracts further into a filament, and only finally collapses fully into a compact halo – the sequence of anisotropic collapse first described by Zel’dovich [13]. This picture can be made quantitative by looking at the local tidal (or velocity-shear) field. Following the tidal-tensor classification reviewed by [13], one computes the Hessian of the (rescaled) gravitational potential ϕ ,

$$T_{\alpha\beta} = \frac{\partial^2 \phi}{\partial x_\alpha \partial x_\beta}, \quad \nabla^2 \phi = \delta, \quad (1)$$

where $\delta = (\rho - \bar{\rho})/\bar{\rho}$ is the dimensionless matter overdensity and ϕ has been normalised by $4\pi G\bar{\rho}$ so that Eq. (1) takes the form of the Poisson equation [13]. Diagonalising the (real, symmetric) tidal tensor at each point gives three eigenvalues $\lambda_1 > \lambda_2 > \lambda_3$; counting how many of them exceed a chosen threshold λ_{th} classifies that point as part of a void (zero eigenvalues above threshold), a sheet (one), a filament (two), or a knot (three) [13]. Voids, sheets, filaments and knots are thus not four independent morphologies but a nested hierarchy, each embedded within the next.

2.1.2 Matter Power Spectrum and Transfer Function

While the tidal-tensor classification above describes the cosmic web *qualitatively*, its statistical properties are quantified through the matter power spectrum $P(k)$. Writing the density contrast in Fourier space as $\delta(\mathbf{k})$ and assuming statistical homogeneity and isotropy, the power spectrum is defined through

$$\langle \delta(\mathbf{k}) \delta^*(\mathbf{k}') \rangle = (2\pi)^3 \delta_D(\mathbf{k} - \mathbf{k}') P(k), \quad (2)$$

where δ_D is the Dirac delta and the angle brackets denote an ensemble average over realisations of the initial conditions [4, 9]. In a simplified form it can be written as

$$P(k) \propto k^{n_s} T^2(k), \quad (3)$$

with n_s the scalar spectral index [9, 5]. The transfer function $T(k)$ can in turn be approximated by

$$T(k) = 1 - \frac{k}{k_{\text{eq}}} + \dots, \quad k_{\text{eq}} = H_0 \sqrt{\frac{2\Omega_m}{\Omega_r}}, \quad (4)$$

[1, 5]. The transfer function is therefore an essential ingredient of the matter power spectrum, alongside the matter and radiation densities. It also depends on the neutrino mass, as illustrated in Figure 1. The three red curves correspond

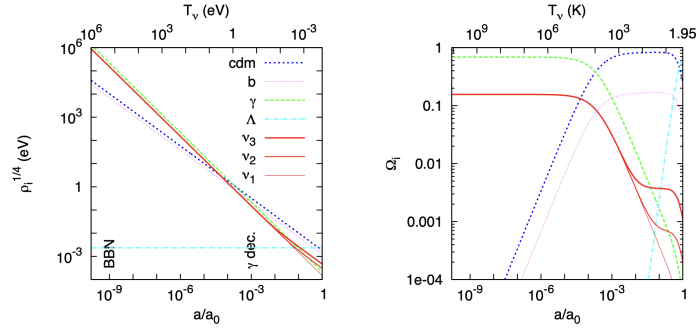


Figure 1: Linear matter transfer function for three neutrino masses (red curves). Reproduced from [11].

to the neutrino masses $m_{\nu_1} = 0$, $m_{\nu_2} = 0.009$ and $m_{\nu_3} = 0.05$ eV [12, 11]. This in turn affects the angular power spectrum C_ℓ , which is defined as

$$C_\ell \approx \int k^2 P(k) dk, \quad (5)$$

and is essentially the Fourier transform of the matter power spectrum [16]. This is fundamental for the present project, since its influence reappears later when generating the initial conditions for the simulations.

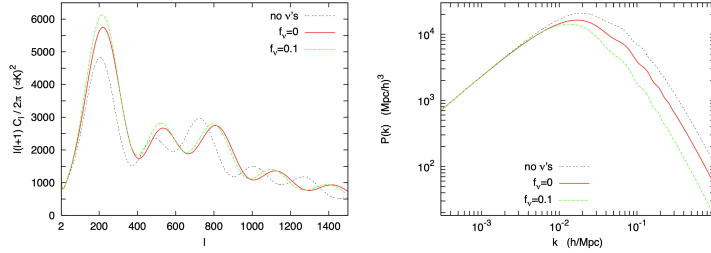


Figure 2: Effect of the neutrino mass on the angular power spectrum. Reproduced from [11].

Obtaining $P(k)$ precisely, including this transfer function, requires solving the coupled linear Einstein–Boltzmann equations for the perturbed Friedmann–Lemaître–Robertson–Walker metric and the phase-space distributions of all matter and radiation species – the same system solved by the differentiable Einstein–Boltzmann module of the Disco-DJ framework used later in this thesis [8].

2.1.3 Lightcones and Snapshots

An N -body simulation naturally outputs *snapshots*: the full particle phase-space (positions and velocities) within a periodic box of fixed comoving side length, at a fixed time. Reproducing how we actually observe the Universe, however, requires a *lightcone*: rather than a single time slice, an observer at a fixed spacetime point receives light emitted from progressively more distant (and hence earlier) shells as one looks to higher redshift, so a simulated lightcone must record particles at the (redshift-dependent) comoving distance from which they are being observed, not at a single global time.

Because the depth of the desired lightcone (out to some $z_{\max}$) is typically larger than a single simulation box, the box is periodically replicated around a centrally placed observer: box replication takes advantage of the fact that a simulation with periodic boundary conditions can, in principle, represent an arbitrarily large volume, at the cost of the same structures reappearing at multiple replica images along a given line of sight [10]. This replication strategy, and the trade-off between box size, structure repetition, and the resulting bias or excess variance in the projected lightcone statistics, is discussed in detail by [10], who additionally introduce a shell-permutation scheme that mitigates the repeated-structure problem by drawing different radial shells of a single lightcone from independent simulations of the same cosmology.

2.2 Cosmological Simulations

Cosmological N -body simulations have historically tracked, and in some respects driven, the development of high-performance computing: the number of particles that can be evolved in a fixed wall-clock time has grown by many orders of magnitude over the past decades, both because of raw hardware improvements and because of continual gains in the efficiency of the gravity solvers themselves [15]. The scientific payoff of these simulations lies in their ability to model the large-scale structure (LSS) of the Universe to the non-linear precision that photometric surveys require. LSS observables are particularly sensitive to the present-day matter density Ω_m and the amplitude of matter clustering σ_8 ; because a lightcone simulation can be sliced into tomographic redshift bins, it further allows the growth of structure to be tracked over cosmic time, which is what makes such simulations sensitive to the dark-energy equation-of-state parameter w_0 (and its possible time evolution) and hence to the physical origin of cosmic acceleration [10].

2.2.1 DISCO-DJ

Disco-DJ (*Differentiable Simulations for Cosmology – Done with JAX*) is a particle-mesh (PM) code whose entire forward model is GPU-accelerated and differentiable, so that gradients of any output with respect to the input parameters – or the initial conditions themselves – are available through automatic differentiation [14]. Like any N -body method it approximates the collisionless dynamics of dark matter, i.e. a solution of the cosmological Vlasov–Poisson system for the phase-space distribution $f(\mathbf{x}, \mathbf{p}, t)$,

$$\frac{\partial f}{\partial t} + \frac{\mathbf{p}}{a^2} \cdot \nabla_{\mathbf{x}} f - \nabla_{\mathbf{x}} \phi \cdot \nabla_{\mathbf{p}} f = 0, \quad \nabla_{\mathbf{x}}^2 \phi = \frac{3\Omega_m H_0^2}{2a} \delta, \quad (6)$$

where $\mathbf{x}$ are comoving coordinates, $\mathbf{p} = a^2 \dot{\mathbf{x}}$ the conjugate momentum, $a(t)$ the scale factor, and ϕ the peculiar gravitational potential sourced by the density contrast δ through the Poisson equation [14].

In the PM scheme the potential is not summed particle-by-particle, as in the tree/FMM solver of Sec. 2.2.2, but obtained on a regular grid: the particles are deposited onto a mesh to give δ , the Poisson equation is solved in Fourier space with a single FFT (where $\nabla^2 \rightarrow -k^2$), and the force $-\nabla_{\mathbf{x}} \phi$ is interpolated back to each particle. A force evaluation is thus global and cheap, at the price of a spatial resolution limited by the mesh spacing – the origin of the small-scale inaccuracy discussed in Sec. 3.1.

Time integration uses a drift–kick–drift leapfrog step. Writing $\mathbf{X}_i^n$ for the position of particle i at step n and $\mathbf{V}_i^n$ for its velocity variable, one step of size

$\tau = \tau_1 + \tau_2$ reads

$$\begin{aligned}\mathbf{X}_i^{n+1/2} &= \mathbf{X}_i^n + \tau_1 \mathbf{V}_i^n, \\ \mathbf{V}_i^{n+1} &= \alpha \mathbf{V}_i^n + \beta \mathbf{A}(\mathbf{X}_i^{n+1/2}), \\ \mathbf{X}_i^{n+1} &= \mathbf{X}_i^{n+1/2} + \tau_2 \mathbf{V}_i^{n+1},\end{aligned}\tag{7}$$

where $\mathbf{A}$ is the PM acceleration evaluated at the half-step position and α, β are coefficients that may depend on time and step size. Different choices of the velocity variable and time coordinate are supported – for example the canonical momentum $\mathbf{V} = a^2 \dot{\mathbf{X}}$, or the growth-time velocity $\mathbf{V} = d\mathbf{X}/dD$ that uses the linear growth factor $D(a)$ itself as the time variable [14].

The key idea exploited by Disco-DJ is that α and β can be chosen so that a single PM step reproduces perturbation theory exactly in its regime of validity. Its default *BullFrog* integrator fixes α, β such that each step follows the second-order Lagrangian perturbation theory (2LPT) trajectory on large scales, with the higher-order truncation error shrinking as the step is refined [14]. Because the large-scale evolution is then correct by construction, per-cent-level accuracy in the present-day power spectrum is reached with only a handful of steps – e.g. about 6 steps at $k \approx 0.2 h/\text{Mpc}$ for 512^3 particles – which is what makes Disco-DJ fast enough to act as the cheap forward model in this work [14].

2.2.2 Pkdgrav

The gravity solver PKDGRAV3 computes the gravitational force with a binary-tree Fast Multipole Method (FMM): rather than only approximating a distant tree cell’s effect on individual particles (as in a Barnes–Hut tree code), it also expands the potential *within* a receiving cell sourced by a distant cell, an interaction that lets the cost of a full force evaluation scale as $\mathcal{O}(N)$ instead of $\mathcal{O}(N \log N)$ [15, Sec. 3.1].

Because particles in different regions of a cosmological box span many orders of magnitude in local density, and hence in dynamical time, a single global time step would be highly wasteful. PKDGRAV3 instead assigns each particle to one of several time-step “rungs”, advancing only the particles active on a given rung at each sub-step, and additionally builds a second, smaller tree restricted to the small subset of particles on the shortest (“very active”) rungs so that this frequently-rebuilt tree stays cheap; the combination is summarised by the kick-drift-kick “umbrella” diagram over one base time step, see Figure 3. This dual-tree multi-stepping scheme is what keeps the tree-construction cost from dominating the runtime even though the shortest rungs may be walked hundreds of times per base step [15, Sec. 3.2].

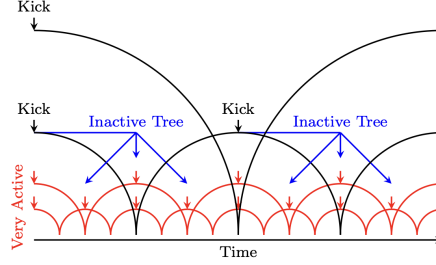


Figure 3: [15]

2.2.3 CosmoGridV1

CosmoGridV1 is a large, publicly released suite of lightcone N -body simulations built specifically for map-level (rather than two-point-function-only) cosmological inference from large-scale-structure probes such as weak lensing and galaxy clustering [10]. It samples a six-dimensional flat w CDM parameter space (Ω_m , σ_8 , w_0 , H_0 , n_s , Ω_b , with a fixed total neutrino mass) at 2500 points arranged on a Sobol sequence, running 7 independent realisations at every point, in addition to 200 realisations (with parameter-derivative pairs) at a fixed fiducial cosmology and a smaller set of benchmark runs at higher resolution used to validate the main configuration’s choices of box size, particle count, and shell thickness [10]. Every simulation is run with PKDGRAV3 in a periodic box of $900 \text{ Mpc}/h$ with 832^3 particles, and stores its lightcone as HEALPix particle-count maps at $N_{\text{side}} = 2048$ out to $z = 3.5$, together with friends-of-friends halo catalogues at each output step [10]. These raw particle-count shells are the basis from which observable maps of weak-lensing convergence, galaxy clustering, and intrinsic alignment are subsequently constructed, and the same dataset has previously been used, together with a shell-based baryon correction model, for a deep-learning cosmological parameter analysis of the KiDS-1000 weak-lensing survey [7].

3 Methods and implementation

3.1 Pipeline Architecture

The reference suite CosmoGridV1 evolves every realisation with PKDGRAV3 (Sec. 2.2.2), whose FMM force solver and adaptive time-stepping resolve structure accurately down to small scales. That accuracy is expensive: each lightcone costs many node-hours, so the millions of realisations required for map-level, simulation-based inference dominate the total cost of the analysis [10, 15].

Disco-DJ offers the opposite trade-off. Its particle-mesh solver and theory-informed BullFrog stepper (Sec. 2.2.1) reproduce the same large-scale clustering

in seconds rather than hours, but the force resolution is capped by the PM grid, so power is progressively lost on small scales – exactly the scales that carry much of the non-Gaussian information exploited by the deep-learning analysis [14].

The aim of this work is therefore to keep Disco-DJ’s speed while recovering the small-scale content that PKDGRAV3 would have produced. Formally, we seek a correction f acting on the cheap lightcone maps such that

$$f(\text{Map}_{\text{DISCO}}) \approx \text{Map}_{\text{COSMOGRID}}, \quad (8)$$

where both maps are generated from the *same* initial conditions and cosmology, so that f has only to model the resolution-dependent difference between the two solvers, not any change of realisation.

Limitations and solutions. Two routes to f were considered. The first is a direct, image-level neural-network correction that maps $\text{Map}_{\text{DISCO}}$ onto $\text{Map}_{\text{COSMOGRID}}$ pixel by pixel. This is expressive, but heavy: full-sky HEALPix maps at $N_{\text{side}} = 2048$ ($\sim 5 \times 10^7$ pixels) are large, live on the sphere rather than on a flat grid, and do not fit on a single GPU without tiling into patches (Sec. 3.4). The second route, adopted here, works in harmonic space and corrects the angular power spectrum C_ℓ of the Disco-DJ maps at high multipole ℓ , where the PM resolution deficit is concentrated. Operating on C_ℓ is far cheaper, is naturally defined on the sphere, and targets the deficit exactly where Eq. (5) relates it to $P(k)$; the trade-off is that it corrects a summary statistic rather than the full field.

3.2 IC generation

3.2.1 Validation against CosmoGridV1

Before using DISCO’s shell builder (Sec. 3.3.1) for any downstream analysis, we first validate that a lightcone produced with our PKDGRAV3 configuration reproduces the reference CosmoGridV1 statistics. To this end we run PKDGRAV3 with its native lightcone module on the exact box, resolution, and cosmology used by CosmoGridV1, and compare the resulting matter angular power spectrum C_ℓ against the corresponding CosmoGridV1 lightcone. Table ?? lists the parameters that must be matched exactly for this comparison to be meaningful, together with the corresponding keys in the PKDGRAV3 configuration file (`cosmology.par`).

3.2.2 Backscaled initial conditions and hand-off to disco

As described in Section 2.1.2, the initial conditions must be built from a correct transfer function evaluated at the correct neutrino mass, since this directly determines whether the resulting field is consistent with CosmoGridV1.

The resulting particle positions and velocities at $a_i = 1/(1 + z_i)$ are exported and used, unmodified, as the initial condition for the corresponding DISCO run: DISCO evolves this state forward to $a = 1$ and accumulates the lightcone with the shell builder of Sec. 3.3.1 over the same redshift range, $z_{\max} = 3.5$ down to $z = 0$, at the same HEALPix resolution $N_{\text{side}} = 2048$.

3.3 Shell Builder

3.3.1 Core

A lightcone output requires the particle distribution to be resampled at every integration step, since the observable past lightcone shell changes continuously as the universe evolves. The most direct implementation stores the full particle snapshot at each of the n_{steps} integration steps and performs the shell decomposition after the simulation has finished. This approach is memory-prohibitive: retaining n_{steps} full-resolution snapshots simultaneously exceeds the available device memory for the particle counts considered here. We therefore interleave the shell decomposition with the time integration, so that at any given time only the particle states immediately before and after the current integration step, together with a set of low-dimensional pixelized count maps, need to be held in memory.

Time discretization. The radial shells are not defined by the integrator itself but are supplied externally, e.g. through a CosmoGridV1-format metainfo table. Each shell $s \in \{1, \dots, S\}$ is specified by a redshift interval $[z_s^{\text{lo}}, z_s^{\text{hi}}]$ and the corresponding comoving-distance interval $[r_s^{\text{lo}}, r_s^{\text{hi}}]$. To ensure that no shell boundary is crossed *within* an integration step, we define the set of integration times as the union of all shell edges,

$$\{z_j\}_{j=0}^S = \text{unique}\left(\{z_s^{\text{lo}}\}_{s=1}^S \cup \{z_s^{\text{hi}}\}_{s=1}^S\right), \quad a_j = \frac{1}{1 + z_j}, \quad (9)$$

sorted in ascending order of a_j (equivalently, descending order of z_j). By construction, every shell boundary coincides exactly with an integration-step boundary. The interval $[a_{\text{ini}}, a_0]$, preceding the first shell, is integrated separately with n_{pre} sub-steps and without shell accumulation, in order to bring the particle state into registry with the shell grid defined in Eq. (9). The remaining $J = S$ intervals $[a_j, a_{j+1}]$, $j = 0, \dots, J - 1$, are each advanced with exactly one integration sub-step.

Accumulation procedure. Algorithm 1 summarizes the per-step accumulation. At step j , the comoving positions and validity mask of all particles are recorded both before the sub-step, $\{\mathbf{x}_p^{(j)}, m_p^{(j)}\}$, and after it, $\{\mathbf{x}_p^{(j+1)}, m_p^{(j+1)}\}$. The scale-factor edges of the step are converted to comoving distance through the cosmology-dependent map $\chi(a)$,

$$R_j = \chi(a_j), \quad R_{j+1} = \chi(a_{j+1}), \quad R_j > R_{j+1}, \quad (10)$$

where the inequality follows from χ being strictly decreasing on $a \in (0, 1]$. The set of shells that are traversed during step j is

$$\mathcal{A}_j = \{s : r_s^{\text{lo}} < R_j \wedge r_s^{\text{hi}} > R_{j+1}\}, \quad (11)$$

i.e. those shells whose radial bin overlaps the annulus $[R_{j+1}, R_j]$ swept out during the step. For each active shell $s \in \mathcal{A}_j$, the pre- and post-step particle states, together with the radial bounds $(r_s^{\text{lo}}, r_s^{\text{hi}})$, are passed to the shell-accumulation routine described in Sec. 3.3.2, which returns a HEALPix pixel count map $\mathbf{M}_s^{(j)} \in \mathbb{N}^{N_{\text{pix}}}$. This map is added to a running total,

$$\mathbf{S}_s += \mathbf{M}_s^{(j)}, \quad s \in \mathcal{A}_j. \quad (12)$$

Because the integration grid is constructed from the shell edges themselves (Eq. 9), $\mathcal{A}_j$ reduces to a single shell in the generic case; the overlap criterion in Eq. (11) is retained in its general form to also cover degenerate or adjacent bins in the input shell table.

Algorithm 1 Streaming shell accumulation

- 1: Load shell table $\{(z_s^{\text{lo}}, z_s^{\text{hi}}, r_s^{\text{lo}}, r_s^{\text{hi}})\}_{s=1}^S$; build integration grid $\{a_j\}_{j=0}^J$ (Eq. 9)
 - 2: Initialize $\mathbf{S}_s \leftarrow \mathbf{0}$ for all s
 - 3: Integrate particle state from a_{ini} to a_0 with n_{pre} sub-steps (no accumulation)
 - 4: **for** $j = 0, \dots, J - 1$ **do**
 - 5: Record pre-step positions and mask, $\{\mathbf{x}_p^{(j)}, m_p^{(j)}\}$
 - 6: Integrate one sub-step: $a_j \rightarrow a_{j+1}$
 - 7: Record post-step positions and mask, $\{\mathbf{x}_p^{(j+1)}, m_p^{(j+1)}\}$
 - 8: Compute R_j, R_{j+1} (Eq. 10) and active shells $\mathcal{A}_j$ (Eq. 11)
 - 9: **for** $s \in \mathcal{A}_j$ **do**
 - 10: $\mathbf{M}_s^{(j)} \leftarrow$ crossing map from $\{\mathbf{x}_p^{(j)}, \mathbf{x}_p^{(j+1)}, m_p^{(j)}, m_p^{(j+1)}\}$ restricted to $[r_s^{\text{lo}}, r_s^{\text{hi}}]$ (Sec. 3.3.2)
 - 11: $\mathbf{S}_s += \mathbf{M}_s^{(j)}$
 - 12: **end for**
 - 13: Discard pre-step state; retain post-step state as pre-step state for $j + 1$
 - 14: **end for**
 - 15: **return** $\{\mathbf{S}_s\}_{s=1}^S$
-

Output. Once all J steps have been processed, the set of per-shell count maps $\{\mathbf{S}_s\}_{s=1}^S$, together with the shell table, is serialized to a single output file. At no point during the procedure are more than $\mathcal{O}(1)$ full particle snapshots and S integer-valued maps of size N_{pix} held simultaneously; this is what removes the memory bottleneck associated with the naive end-of-run decomposition.

3.3.2 Crossing Condition

The condition follows the moving-lightcone formulation of PKDGRAV3 [6] exactly, rather than the vector-trajectory intersection used in the earlier sketch.

Periodic replicas. The observer is placed at the box centre, $\mathbf{o} = (L/2, L/2, L/2)$, with L the box side length. Since the requested lightcone depth $\chi_{\max} = \chi(1/(1+z_{\max}))$ generally exceeds L , the simulation volume is periodically replicated: let

$$\mathcal{D} = \left\{ \mathbf{d} = (n_1, n_2, n_3) L : \mathbf{n} \in \mathbb{Z}^3, \text{ replicated cube intersects the ball of radius } \chi_{\max} \text{ about } \mathbf{o} \right\} \quad (13)$$

denote the finite set of replica offsets, precomputed once from $(\chi_{\max}, L)$. For each $\mathbf{d} \in \mathcal{D}$ we also precompute the nearest and farthest possible distance of the replicated cube from the observer,

$$r_{\min}(\mathbf{d}) = \|\max(0, |\mathbf{d}| - L/2)\|, \quad r_{\max}(\mathbf{d}) = \|\mathbf{d}| + L/2\|, \quad (14)$$

(componentwise max and absolute value, followed by the Euclidean norm), which allow a replica to be rejected for a given step without evaluating any particle if its radial range cannot overlap the step's lightcone annulus.

Linear crossing formula. For step j with N-body boundaries $a_j < a_{j+1}$, define, as before, $r_j^{\text{hi}} = \chi(a_j)$ and $r_j^{\text{lo}} = \chi(a_{j+1})$. For a particle with pre-/post-step positions $\mathbf{x}_p(a_j), \mathbf{x}_p(a_{j+1})$ relative to the box centre, and a replica offset $\mathbf{d} \in \mathcal{D}$, define the shifted positions and their radii

$$\mathbf{R}_0 = \mathbf{x}_p(a_j) + \mathbf{d}, \quad \mathbf{R}_1 = \mathbf{x}_p(a_{j+1}) + \mathbf{d}, \quad r_0 = \|\mathbf{R}_0\|, \quad r_1 = \|\mathbf{R}_1\|. \quad (15)$$

Rather than solving for the intersection of the interpolated *vector* trajectory with the receding lightcone sphere (which requires a quadratic root), the implementation follows PKDGRAV3 and linearly interpolates the *scalar* radii of the particle and of the lightcone front over the step,

$$r_p(v) = r_0 + v(r_1 - r_0), \quad R(v) = r_j^{\text{hi}} - v(r_j^{\text{hi}} - r_j^{\text{lo}}), \quad v \in [0, 1], \quad (16)$$

and solves $r_p(v^*) = R(v^*)$ for the crossing parameter,

$$v^* = \frac{r_j^{\text{hi}} - r_0}{(r_j^{\text{hi}} - r_j^{\text{lo}}) + (r_1 - r_0)}. \quad (17)$$

A particle–replica pair is accepted only if $v^* \in [0, 1]$; since Eq. (17) partitions the step interval, this guarantees that every particle is caught in *at most one* step, for a fixed replica – evaluating the crossing test against the (generally narrower) metainfo shell bounds instead of the full step bounds $[r_j^{\text{lo}}, r_j^{\text{hi}})$ would break this partition and could double-count particles moving across two adjacent shells within the same step. Pairs with $|r_1 - r_0|$ below a numerical tolerance (10^{-20}) are treated as non-moving and rejected, and the denominator of Eq. (17) is floored in magnitude at 10^{-10} to avoid division by a near-zero value.

Shell assignment and pixelization. The crossing position is obtained by applying v^* to the *vector* positions,

$$\mathbf{x}_{\text{cross}} = (1 - v^*) \mathbf{R}_0 + v^* \mathbf{R}_1, \quad r_{\text{cross}} = \|\mathbf{x}_{\text{cross}}\|, \quad (18)$$

and the pair is finally kept only if its crossing radius falls inside the metainfo (or, in the absence of an override, the step) shell bounds,

$$r_s^{\text{lo}} \leq r_{\text{cross}} < r_s^{\text{hi}}. \quad (19)$$

Surviving crossings are pixelized with the equal-area HEALPix ring scheme at resolution N_{side} from the unit vector $\hat{\mathbf{x}}_{\text{cross}} = \mathbf{x}_{\text{cross}}/r_{\text{cross}}$, and the corresponding pixel of the shell count map is incremented by one. Algorithm 2 summarizes the full test, which is applied to every particle and every replica $\mathbf{d} \in \mathcal{D}$ surviving the fast reject of Eq. (14), for every step j and every active shell $s \in \mathcal{A}_j$.

Algorithm 2 Lightcone crossing test (per step j , per replica $\mathbf{d}$)

Require: positions $\mathbf{x}_p(a_j), \mathbf{x}_p(a_{j+1})$; step radii $r_j^{\text{lo}}, r_j^{\text{hi}}$; shell bounds $r_s^{\text{lo}}, r_s^{\text{hi}}$; replica $\mathbf{d}$

- 1: $\mathbf{R}_0 \leftarrow \mathbf{x}_p(a_j) + \mathbf{d}, \quad \mathbf{R}_1 \leftarrow \mathbf{x}_p(a_{j+1}) + \mathbf{d}$
- 2: $r_0 \leftarrow \|\mathbf{R}_0\|, \quad r_1 \leftarrow \|\mathbf{R}_1\|$
- 3: **if** $|r_1 - r_0| \leq \epsilon_{\text{move}}$ **then**
- 4: **return reject** (*stationary*)
- 5: **end if**
- 6: $v^* \leftarrow (r_j^{\text{hi}} - r_0) / [(r_j^{\text{hi}} - r_j^{\text{lo}}) + (r_1 - r_0)] \triangleright$ Eq. 17, denominator floored at ϵ_{den}
- 7: **if** $v^* \notin [0, 1]$ **then**
- 8: **return reject** (*not caught this step*)
- 9: **end if**
- 10: $\mathbf{x}_{\text{cross}} \leftarrow (1 - v^*) \mathbf{R}_0 + v^* \mathbf{R}_1, \quad r_{\text{cross}} \leftarrow \|\mathbf{x}_{\text{cross}}\|$
- 11: **if** $r_{\text{cross}} \notin [r_s^{\text{lo}}, r_s^{\text{hi}}]$ **then**
- 12: **return reject** (*wrong shell*)
- 13: **end if**
- 14: $\text{pix} \leftarrow \text{VEC2PIX}(N_{\text{side}}, \hat{\mathbf{x}}_{\text{cross}})$
- 15: $\mathbf{S}_s[\text{pix}] += 1$

This procedure reproduces the corner-observer, box-replication scheme of PKDGRAV3 [6] almost exactly, with two differences: the observer is placed at the box centre rather than a corner, and the replica set $\mathcal{D}$ (Eq. 13) is sized adaptively from the requested depth χ_{max} and box length L – together with a per-step, per-replica radial reject (Eq. 14) – rather than fixed a priori to $6^3 = 216$ images.

3.4 Neural Network

3.4.1 Dimensional Reduction

VRAM issues - $\hat{\mathbf{i}}$ patches or in spherical harmonics

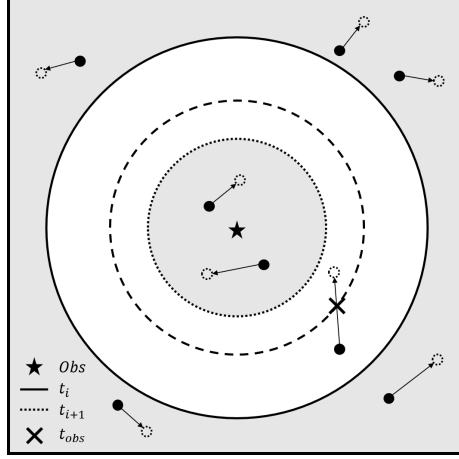


Figure 4: [6]

3.4.2 Data preparation

3.4.3 Network Architecture

3.4.4 Training Procedure

3.4.5 Validation Strategy

(Angular) Power Spectru, Kappa - δ UFALCON

4 Computational Infrastructure

4.1 Computing Environments

Development proceeded in stages of increasing scale, from a local workstation used for prototyping to two GPU clusters used for production runs.

4.1.1 Local Development

Both DISCO and PKDGRAV3 were first set up on a local machine, to establish a minimal working configuration and to inspect their output interactively. Running locally shortens the debugging loop considerably: results can be plotted and checked immediately, without the queueing latency of a batch system. Small tests used a resolution of $16^3 = 4096$ particles, which completes in under a minute on a single core. The binding constraint is memory: storing only the phase-space state (three positions and three velocities per particle, in single precision) already for a CosmoGridV1-sized run,

$$832^3 \approx 5.8 \times 10^8 \text{ particles} \times 24 \text{ bytes} \approx 14 \text{ GB},$$

exceeds the RAM of a typical workstation before any tree, mesh, or lightcone buffers are added. Production resolutions therefore require a cluster.

4.1.2 Euler (ETH Zürich)

On ETH’s Euler cluster we built a separate virtual environment for each code, because DISCO and PKDGRAV3 have conflicting library dependencies that cannot coexist in a single environment. Euler provides the memory and the GPU partitions needed to run at the full 832^3 resolution, with substantial speed-ups from the GPU-accelerated builds. Depending on the configuration a single simulation can still be long-running, so jobs are distributed across multiple nodes where the code supports it.

4.1.3 CSCS (Alps / Piz Daint)

The largest runs use the CSCS Alps system, whose NVIDIA GH200 nodes provide 120 GB of on-package memory. This large VRAM is well suited to scaling Disco-DJ up and to shortening the wall-clock time of each run, whether generating the training pairs or, later, training the network itself.

4.2 Software Engineering

4.2.1 Code Structure

Executables and virtual environments are kept in the cluster `HOME` directory, which is persistent and backed up. Because `HOME` has a strict quota, all bulk output is written instead to `SCRATCH`: simulation snapshots and shell maps, per-job logs, and diagnostic plots of snapshots and shells. Keeping the full log of every submission proved valuable both for debugging and for later cross-run comparison. The CosmoGridV1 reference data (~ 110 TB) was transferred to the CSCS entry point with a dedicated Globus transfer job.

4.2.2 Job Scheduling and Submission

Every Python script is paired with a Slurm batch script. The Python entry point is written to run unchanged both locally and on the cluster, while the accompanying batch script requests the resources and captures the output. The same job can then be launched interactively (`bash`, for fast monitoring directly in the terminal) or submitted to the queue (`sbatch`, to run detached and log to file).

Each batch script follows the same two-part layout. The first part is a header of `#SBATCH` directives that declares the resource request to the scheduler, and the second is a short execution body that prepares the environment and launches the driver. The header groups naturally into three concerns:

- **Resources** – how much hardware to reserve: the number of `nodes`, the GPUs per node (`gres=gpu`), the task layout (`ntasks-per-node`), `exclusive` node access, and a wall-clock `time` limit after which the job is killed.

- **Accounting** – where the allocation is charged and queued: the `account` and the `partition`.
- **Bookkeeping** – where results and diagnostics go: a human-readable `job-name`, the working directory (`chdir`), and the `output/error` log paths, where `%j` expands to the unique job ID so that concurrent runs never overwrite one another.

A representative submission script is shown below.

```
#!/bin/bash
# --- Resources ---
#SBATCH --nodes=2
#SBATCH --exclusive
#SBATCH --ntasks-per-node=1
#SBATCH --gres=gpu:4           # GPUs per node
#SBATCH --time=01:00:00       # wall-clock limit (hh:mm:ss)
# --- Accounting ---
#SBATCH --account=sk037
#SBATCH --partition=normal
# --- Bookkeeping ---
#SBATCH --job-name=discodj-custom
#SBATCH --chdir=/capstor/scratch/cscs/damrein/outputs/disco_custom
#SBATCH --output=/capstor/scratch/cscs/damrein/outputs/logs/disco_custom/slurm-%j.out
#SBATCH --error=/capstor/scratch/cscs/damrein/outputs/logs/disco_custom/slurm-%j.err

# --- Execution body ---
# 1. activate the code-specific virtual environment
# 2. define paths and simulation parameters
# 3. run the Python driver
```

Because only the header changes between runs, adapting a job to a new configuration – more nodes, a longer time limit, or a different output directory – amounts to editing a few directives, while the Python driver is reused unchanged.

4.2.3 Version Control (GitHub)

The whole project – the analysis scripts together with the modified DISCO that includes the shell builder – is kept in a dedicated Git repository. Beyond the usual protection against machine failure, this gives a reliable way to return to a known-good state when an experimental change breaks the pipeline, and keeps the many configuration variants organised.

5 Results

from [3]

6 Conclusion

References

- [1] J. M. Bardeen et al. “The Statistics of Peaks of Gaussian Random Fields”. In: *The Astrophysical Journal* 304 (May 1986), p. 15. ISSN: 0004-637X. DOI: 10.1086/164143. URL: <https://ui.adsabs.harvard.edu/abs/1986ApJ...304...15B/abstract> (visited on 07/07/2026).
- [2] J. Bond, Lev Kofman, and D. Pogosyan. “How filaments of galaxies are woven into the cosmic web”. In: *Nature* 380 (Apr. 18, 1996), pp. 603–606. DOI: 10.1038/380603a0.
- [3] Michaël Defferrard et al. *DeepSphere: a graph-based spherical CNN*. Dec. 30, 2020. DOI: 10.48550/arXiv.2012.15000. arXiv: 2012.15000[cs.LG]. URL: <http://arxiv.org/abs/2012.15000> (visited on 07/02/2026).
- [4] Scott Dodelson and Fabian Schmidt. “Modern Cosmology”. In: *Modern Cosmology* (2020). ISBN: 9780128159491. DOI: 10.1016/C2017-0-01943-2. URL: <https://ui.adsabs.harvard.edu/abs/2020moco.book.....D/abstract> (visited on 07/04/2026).
- [5] Daniel J. Eisenstein and Wayne Hu. “Baryonic Features in the Matter Transfer Function”. In: *The Astrophysical Journal* 496.2 (Apr. 1998), pp. 605–614. ISSN: 0004-637X, 1538-4357. DOI: 10.1086/305424. arXiv: astro-ph/9709112. URL: <http://arxiv.org/abs/astro-ph/9709112> (visited on 07/07/2026).
- [6] `example/pkdgrav3-lightcone · main · Janis Fluri / PkDgrav3_intro · GitLab`. GitLab. Feb. 2, 2022. URL: https://cosmo-gitlab.phys.ethz.ch/jafluri/pkdgrav3_intro/-/tree/main/example/pkdgrav3_lightcone (visited on 07/04/2026).
- [7] Janis Fluri et al. “A Full Λ CDM Analysis of KiDS-1000 Weak Lensing Maps using Deep Learning”. In: *Physical Review D* 105.8 (Apr. 20, 2022), p. 083518. ISSN: 2470-0010, 2470-0029. DOI: 10.1103/PhysRevD.105.083518. arXiv: 2201.07771[astro-ph.CO]. URL: <http://arxiv.org/abs/2201.07771> (visited on 07/02/2026).
- [8] Oliver Hahn, Florian List, and Natalia Porqueres. “DISCO-DJ I: a differentiable Einstein-Boltzmann solver for cosmology”. In: *Journal of Cosmology and Astroparticle Physics* 2024.6 (June 1, 2024), p. 063. ISSN: 1475-7516. DOI: 10.1088/1475-7516/2024/06/063. arXiv: 2311.03291[astro-ph.CO]. URL: <http://arxiv.org/abs/2311.03291> (visited on 07/02/2026).
- [9] Dragan Huterer. “Growth of Cosmic Structure”. In: *The Astronomy and Astrophysics Review* 31.1 (Dec. 2023), p. 2. ISSN: 0935-4956, 1432-0754. DOI: 10.1007/s00159-023-00147-4. arXiv: 2212.05003[astro-ph.CO]. URL: <http://arxiv.org/abs/2212.05003> (visited on 07/07/2026).

- [10] Tomasz Kacprzak et al. “CosmoGridV1: a simulated Λ CDM theory prediction for map-level cosmological inference”. In: *Journal of Cosmology and Astroparticle Physics* 2023.2 (Feb. 1, 2023), p. 050. ISSN: 1475-7516. DOI: 10.1088/1475-7516/2023/02/050. arXiv: 2209.04662[astro-ph.CO]. URL: <http://arxiv.org/abs/2209.04662> (visited on 07/02/2026).
- [11] Julien Lesgourgues and Sergio Pastor. “Massive neutrinos and cosmology”. In: *Physics Reports* 429.6 (July 2006), pp. 307–379. ISSN: 03701573. DOI: 10.1016/j.physrep.2006.04.001. arXiv: astro-ph/0603494. URL: <http://arxiv.org/abs/astro-ph/0603494> (visited on 07/07/2026).
- [12] Julien Lesgourgues and Sergio Pastor. “Neutrino mass from Cosmology”. In: *Advances in High Energy Physics* 2012 (2012), pp. 1–34. ISSN: 1687-7357, 1687-7365. DOI: 10.1155/2012/608515. arXiv: 1212.6154[hep-ph]. URL: <http://arxiv.org/abs/1212.6154> (visited on 07/07/2026).
- [13] Noam I. Libeskind et al. “Tracing the cosmic web”. In: *Monthly Notices of the Royal Astronomical Society* 473.1 (Jan. 1, 2018), pp. 1195–1217. ISSN: 0035-8711, 1365-2966. DOI: 10.1093/mnras/stx1976. arXiv: 1705.03021[astro-ph.CO]. URL: <http://arxiv.org/abs/1705.03021> (visited on 07/04/2026).
- [14] Florian List et al. “DISCO-DJ II: a differentiable particle-mesh code for cosmology”. In: *Journal of Cosmology and Astroparticle Physics* 2026.6 (June 1, 2026), p. 033. ISSN: 1475-7516. DOI: 10.1088/1475-7516/2026/06/033. arXiv: 2510.05206[astro-ph.CO]. URL: <http://arxiv.org/abs/2510.05206> (visited on 07/02/2026).
- [15] Douglas Potter, Joachim Stadel, and Romain Teyssier. *PKDGRAV3: Beyond Trillion Particle Cosmological Simulations for the Next Era of Galaxy Surveys*. arXiv.org. Sept. 27, 2016. URL: <https://arxiv.org/abs/1609.08621v1> (visited on 07/03/2026).
- [16] Uros Seljak and Matias Zaldarriaga. “A Line of Sight Approach to Cosmic Microwave Background Anisotropies”. In: *The Astrophysical Journal* 469 (Oct. 1996), p. 437. ISSN: 0004-637X, 1538-4357. DOI: 10.1086/177793. arXiv: astro-ph/9603033. URL: <http://arxiv.org/abs/astro-ph/9603033> (visited on 07/07/2026).